

Università degli Studi di Salerno

Corso di Ingegneria del Software

MediBook
TP
Versione 1.4



Data: 11/01/2026

Progetto: MediBook	Versione: 1.4
Documento: TP	Data: 11/01/2026

Coordinatore del progetto:

Nome	Matricola
Lelio Crisci	0512119092

Partecipanti:

Nome	Matricola
Samuele Colucci	0512119140
Christian Casale	0512119842

Scritto da:	Christian Casale
--------------------	------------------

Revision History

Data	Versione	Descrizione	Autore
18/12/2025	1.1	Aggiunta template e inizio stesura Suspension and resumption	Samuele Colucci
21/12/2025	1.2	Aggiunta test case	Samuele Colucci
21/12/2025	1.3	Revisione e correzione documento	Christian Casale
11/01/2026	1.4	Aggiunta Test Case	Lelio Crisci

Contents

1. Introduction	4
2. Relationship to other documents	4
3. System overview	4
4. Features to be tested/not to be tested	5
5. Pass/Fail criteria	6
6. Approach	6
6.1 Testing di unità	6
6.2 Testing di integrazione	6
6.3 Testing di sistema	7
7. Suspension and resumption	7
7.1 Criteri di Sospensione	7
7.2 Criteri di Ripresa	7
8. Testing materials (hardware/software requirements)	8
Hardware	8
Software	8
9. Test cases	9
9.1 Funzionalità: Autenticazione (UC1)	9
9.2 Funzionalità: Registrazione Account (UC2)	10
9.3 Funzionalità: Prenotazione Visita (UC3)	12
9.4 Funzionalità: Modifica Prenotazione – SegreteriaPrenotazioni (UC4)	13
9.5 Funzionalità: Gestione Visita (UC6)	13
9.6 Funzionalità: Primo Accesso con Cambio Password Obbligatorio (UC8)	14
9.7 Funzionalità: Compilazione e Salvataggio Referto (UC9)	16

Test Plan

1. Introduction

La presente sezione descrive gli obiettivi e l'estensione delle attività di test per il sistema MediBook. L'obiettivo è fornire un quadro di riferimento per la verifica della conformità del software rispetto ai requisiti specificati nel RAD e nell'SDD.

2. Relationship to other documents

Il piano di test fa riferimento ai seguenti documenti:

- RAD: Per la verifica dei requisiti funzionali.
- ODD: Per la verifica dei vincoli sulle classi e sui dati.

Naming Scheme: I casi di test saranno identificati univocamente con la stringa TC_<Funzione>_<Numero> (es. TC_PRE_1).

3. System overview

Il sistema oggetto di test è suddiviso nei seguenti componenti logici, testati secondo una granularità che va dal singolo metodo al flusso completo:

- **Presentation Layer (JSP/HTML):** Interfaccia utente basata su JavaServer Pages. I test verificano la corretta visualizzazione dei dati e l'invio dei form verso i Controller.
- **Business Logic Layer (Spring Service):** Il cuore del sistema dove risiedono i vincoli **OCL**. I test di unità si concentrano su queste classi (es. GestionePrenotazioni) per validare la logica di business.
- **Data Access Layer (Spring Data JPA):** Gestisce la comunicazione con il DB MySQL tramite le Repository. Viene testata l'integrità della persistenza.

4. Features to be tested/not to be tested

Funzionalità da testare:

Gestione Autenticazione e Sicurezza: Verifica del Login con ruoli differenziati (Paziente, Medico, Segreteria).

- Controllo della corretta cifratura delle password tramite **PasswordService**.
- Verifica del flusso di "Cambio Password Obbligatorio" al primo accesso.

Gestione Prenotazioni:

- Validazione dei vincoli temporali (impedire prenotazioni in date passate).
- Verifica della disponibilità degli slot orari (evitare sovrapposizioni per lo stesso medico).
- Modifica e annullamento delle prenotazioni esistenti con aggiornamento dello stato.

Gestione Referti e Diagnostica:

- Verifica del caricamento referti solo per visite in stato "EFFETTUATA".
- Transizione automatica dello stato della prenotazione a "CONCLUSA" dopo il caricamento.
- Visualizzazione sicura del referto da parte del paziente proprietario.

Gestione Utenza:

- Registrazione di nuovi pazienti con controllo di unicità su Email e Codice Fiscale.

Funzionalità non testate:

Le seguenti aree non rientrano nell'ambito di questo piano di test per le motivazioni indicate:

- **Performance e Stress Test:** Non verrà testata la risposta del database o del server sotto carichi di traffico estremi, in quanto il sistema è dimensionato per un uso clinico standard.
- **Compatibilità Browser Obsoleti:** Il test è limitato ai browser moderni (Chrome, Firefox, Edge); non è garantito né testato il supporto per Internet Explorer o versioni legacy.
- **Infrastruttura di Terze Parti:** Non vengono testate le funzionalità native del framework **Spring Boot** o di **Hibernate**, assumendone l'affidabilità come librerie standard.
- **Sicurezza di Rete:** Non sono oggetto di test la configurazione dei certificati SSL/TLS o la protezione contro attacchi DDoS a livello infrastrutturale.

5. Pass/Fail criteria

Pass: L'output effettivo coincide con l'output atteso e non vengono sollevate eccezioni non gestite.

Fail: L'output differisce, il sistema va in crash, o viene mostrato un errore non user-friendly.

6. Approach

L'attività di testing del sistema MediBook è stata pianificata seguendo una strategia **Bottom-Up** incrementale. Questo approccio prevede la suddivisione del processo in tre livelli logici sequenziali, partendo dalla verifica rigorosa dei singoli componenti isolati (White-Box Testing) fino alla validazione dei flussi funzionali completi attraverso l'interfaccia utente (Black-Box Testing).

L'obiettivo è identificare e risolvere i difetti logici alla radice prima che questi si propaghino ai livelli superiori dell'architettura.

6.1 Testing di unità

In questa fase vengono testate le singole classi in isolamento. L'obiettivo è validare la logica interna e il rispetto dei contratti formali definiti nell'ODD.

- **Oggetto del test:**
 - **Classi Entity (it.unisa.medibook.model):** Verifica della corretta creazione degli oggetti del dominio e delle loro associazioni.
 - **Service Layer (it.unisa.medibook.service):** È il cuore del test di unità. Qui vengono verificati i **vincoli OCL** (pre e post-condizioni) per ogni metodo (es. `modificaPrenotazione`, `checkPassword`).
- **Obiettivo:** Assicurarsi che ogni modulo esegua correttamente il suo compito atomico e gestisca correttamente le eccezioni in caso di violazione dei vincoli (es. inserimento data passata).

6.2 Testing di integrazione

Questa fase verifica la corretta comunicazione tra i moduli precedentemente testati, concentrandosi sullo scambio dati tra i diversi layer dell'architettura Spring Boot.

- **Oggetto del test:** Interazione tra **Controller** e **Service**.
 - Interazione tra **Service** e **Repository** (it.unisa.medibook.modelStorage).
- **Obiettivo:** Verificare che le transazioni sul database (gestite tramite Spring Data JPA) siano atomiche e coerenti, e che il passaggio di dati tra il livello di controllo e quello di persistenza

non alteri l'integrità delle informazioni

6.3 Testing di sistema

In questa fase finale viene verificato il comportamento globale del sistema attraverso l'interfaccia utente, simulando le azioni reali degli utenti finali (Paziente, Medico, Segreteria).

- **Oggetto del test:** Interfaccia Web (JSP/JavaScript) integrata con il Backend.
- **Obiettivo:** Validare la conformità del software rispetto ai **Casi d'Uso (UC)** descritti nel RAD. Si verifica che l'utente possa completare i flussi principali (es. prenotare una visita, ricevere l'email, visualizzare il referto) senza errori di navigazione o di logica.

7. Suspension and resumption

Questa sezione definisce i criteri per sospendere e riprendere le attività di test.

7.1 Criteri di Sospensione

Il testing sarà sospeso se si verifica una delle seguenti condizioni:

- Vengono scoperti bug critici (Blocking Bugs) che impediscono l'utilizzo delle funzioni principali (es. Login non funzionante).
- L'ambiente di test (Database o Server) non è disponibile o è instabile.
- Il codice rilasciato è incompleto rispetto alle specifiche previste per la fase corrente.

7.2 Criteri di Ripresa

Abbiamo previsto delle modifiche future al sistema dopo il rilascio.

Pertanto sarà necessario, dopo aver introdotto i cambiamenti, testare le nuove componenti: se esse introducono dei fault che impattano sulle componenti già esistenti, allora verranno testate nuovamente anche quest'ultime Testing materials (hardware/software requirements) attraverso il **Testing di Regression**.

8. Testing materials (hardware/software requirements)

Per eseguire i test del sistema MediBook è sufficiente una sola postazione di lavoro (PC o Laptop) configurata come segue:

Hardware

- **Computer:** Un comune PC portatile (Windows o Mac) utilizzato sia per la fase di sviluppo che per l'esecuzione dei test di integrazione.

Software

- **Sistema Operativo:** Windows 10/11 o macOS.
- **Database:** MySQL Server 8.0 (installato localmente), per la gestione dei dati persistenti durante i test.
- **Ambiente Java:** JDK 17 (o superiore), necessario per far girare il backend Spring Boot.
- **Ambiente Web:** Apache Tomcat (integrato in Spring Boot), per il rendering delle pagine JSP.
- **Browser:** Google Chrome o Firefox per i test di sistema (interfaccia utente).
- **Strumenti:** Un IDE (es. IntelliJ IDEA, Eclipse o NetBeans) per l'esecuzione dei test JUnit 5 e la verifica dei vincoli OCL.

9. Test cases

In questa sezione vengono definiti i casi di test pianificati. Per garantire una copertura completa delle situazioni critiche, è stato utilizzato il metodo **Category Partition**, scomponendo gli input in categorie e scelte rappresentative.

9.1 Funzionalità: Autenticazione (UC1)

Verifica che il sistema gestisca correttamente l'accesso di utenti registrati e non.
Definizione dei Parametri e Categorie:

Parametro: Email [E]

- Email presente nel DB [propertyEmailOk]
- Email non presente [error]

Parametro: Password [P]

- Password corretta (corrispondente all'email) [propertyPassOk]
- Password errata [error]

ID Test Case	Combinazione	Descrizione Scenario	Esito Atteso
TC_AUC_1	E:2, P:2	Utente inserisce email inesistente e password errata.	Email o password errata
TC_AUC_2	E:1, P:2	Utente inserisce email corretta ma password errata.	Email o password errata
TC_AUC_3	E:1, P:1	Utente inserisce credenziali corrette.	Redirect alla HomePage

9.2 Funzionalità: Registrazione Account (UC2)

Si verifica che il sistema accetti la registrazione solo se tutti i campi obbligatori (Email, Password, Nome Utente, Codice Fiscale) rispettano i formati previsti e i vincoli di unicità nel database.

Parametro: Formato Email [F]

- `^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$` Formato valido [propertyEmailOk]
- Formato non valido [error]

Parametro: Presenza Email nel Database [D]

- Email nuova (Non presente) [propertyNewEmail]
- Email duplicata (Già presente) [error]

Parametro: Codice Fiscale [CF]

- `^[A-Z]{6}[0-9]{2}[A-Z][0-9]{2}[A-Z][0-9]{3}[A-Z]$` Formato valido (16 caratteri alfanumerici corretti) [propertyCFOk]
- Formato errato (lunghezza o caratteri errati) [error]

Parametro: Password [PW]

- Lunghezza >8 and <20 [propertyLengthPWOk]
- Lunghezza <8 or >20 [error]

Codice Test	Combinazione	Descrizione Scenario	Esito Atteso
TC_REG_1	F:2, D:1, CF:1, PW:1	L'utente inserisce tutti i dati corretti, ma l'email ha un formato errato.	Formato email non valido

Codice Test	Combinazione	Descrizione Scenario	Esito Atteso
TC_REG_2	F:1, D:2, CF:1, PW:1	L'utente inserisce dati corretti, ma l'email esiste già nel sistema.	Email già registrata
TC_REG_3	F:1, D:1, CF:2, PW:1	L'utente inserisce tutti i dati corretti, ma il Codice Fiscale è errato (es. lunghezza diversa da 16 o caratteri invalidi).	Codice Fiscale non valido
TC_REG_4	F:1, D:1, CF:1, PW:2	Password fuori range (minore di 8 o maggiore di 20 caratteri).	Lunghezza Password non valida
TC_REG_5	F:1, D:1, CF:1, PW:1	Tutti i dati rispettano formati, lunghezze e unicità.	Registrazione completata con successo

9.3 Funzionalità: Prenotazione Visita (UC3)

Verifica la logica di prenotazione, assicurando che non si possano creare sovrapposizioni o prenotazioni inconsistenti.

Definizione dei Parametri e Categorie:

Parametro: Data Visita [D]

- Data Futura (es. domani) [propertyDataOk]
- Data Passata [error]

Parametro: Disponibilità Medico/Slot [S]

- Slot orario libero [propertySlotOk]
- Slot orario già occupato da altra visita [error]

Parametro: Stato Paziente [U]

- Paziente Attivo [propertyUserOk]
- Paziente Sospeso/Non Loggato [error]

ID Test Case	Combinazione	Descrizione Scenario	Esito Atteso
TC_PRE_1	D:2, S1, U1	Tentativo di prenotazione nel passato, medico tecnicamente disponibile e paziente attivo.	Data non valida
TC_PRE_2	D:1, S:2, U1	Tentativo di prenotazione su orario corretto, medico occupato, paziente attivo.	Slot non disponibile
TC_PRE_3	D:1, S:1, U:2	Utente non loggato tenta di prenotare orario disponibile, slot libero.	Richiesta Login
TC_PRE_4	D:1, S:1, U:1	Utente attivo prenota in data valida e slot libero.	Prenotazione Confermata

9.4 Funzionalità: Modifica Prenotazione – Segreteria Prenotazioni (UC4)

Si verifica che un operatore di Segreteria possa modificare data e ora di una prenotazione esistente, garantendo che l'operazione sia permessa solo su visite future e verso orari disponibili.

Parametro: Prenotazione Originale [ID]

- Esistente e Modificabile (Data futura) [propertyIdOk]
- Data Passata [error]

Parametro: Disponibilità Nuovo Slot [S]

- Slot Libero [propertySlotFree]
- Slot Occupato (Già prenotato da altri) [error]

Codice Test	Combinazione	Descrizione Scenario	Esito Atteso
TC_MOD_1	ID:2, S:1	Tentativo di modifica di una visita con ID errato, verso un nuovo orario valido.	Id non presente
TC_MOD_2	ID:1, S:2	Tentativo di spostamento su uno slot orario già occupato da un altro paziente, con id corretto.	Orario non disponibile
TC_MOD_3	ID:2, S:1	Si tenta di modificare una visita la cui data originale è nel passato.	Impossibile modificare una visita conclusa
TC_MOD_4	ID:1, S:1	Si tenta di modificare una prenotazione esistente e pianificata per il futuro	Prenotazione modificata

9.5 Funzionalità: Gestione Visita (UC6)

Questa sezione verifica la logica funzionale del cambio di stato. Il sistema deve permettere la modifica solo se la visita non è già stata processata.

Parametro: Stato Iniziale della Visita [S]

- [propertyStateOpen] S1: Stato "PRENOTATA". La visita è in programma e può essere modificata.
- [error] S2: Stato diverso da "PRENOTATA", ovvero "CONCLUSA" o "ANNULLATA". La visita è storicizzata e immutabile.

Parametro: Azione Richiesta [A]

- [actionExecute] A1: Marcare la visita come "EFFETTUATA" (per abilitare il referto).
- [actionCancel] A2: Marcare la visita come "ANNULLATA" (il paziente non si è presentato o visita disdetta).

Codice Test	Combinazione	Descrizione Scenario	Esito Atteso
TC_VIS_1	S1, A1	Tentativo di esecuzione della visita da parte del medico	Lo stato della visita diventa "EFFETTUATA" .
TC_VIS_2	S1, A2	Tentativo di annullamento della visita	Lo stato della visita diventa "ANNULLATA"
TC_VIS_3	S2, A1	Tentativo di eseguire un'azione su una visita già "CONCLUSA" o "ANNULLATA"	Impossibile modificare una visita già processata

9.6 Funzionalità: Primo Accesso con Cambio Password Obbligatorio (UC8)

Si verifica che, al primo login, il sistema forzi l'utente a modificare la password provvisoria. Il sistema deve validare la vecchia password, controllare la robustezza della nuova e assicurarsi che la conferma coincida

Parametro: Vecchia Password (Provvisoria) [VP]

- [propertyOldPassOk] VP1: La password inserita corrisponde esattamente a quella provvisoria salvata nel DB (o inviata via mail).
- [error] VP2: Password errata (non corrisponde all'hash nel DB).

Parametro: Nuova Password (Formato) [NP]

- [propertyNewPassOk] NP1: Formato valido (Lunghezza 8-20 caratteri).
- [error] NP2: Formato non valido (Troppo corta o caratteri non ammessi).
- [error] NP3: La nuova password è identica alla vecchia

Parametro: Conferma Password [CP]

- [propertyConfirmOk] CP1: La stringa inserita nel campo "Conferma" è identica alla "Nuova Password".
- [error] CP2: Le due stringhe non coincidono.

Codice Test	Combinazione	Descrizione Scenario	Esito Atteso
TC_CPW_1	VP1, NP1, CP1	L'utente compila i campi correttamente	Successo nel cambio password.
TC_CPW_2	VP1, NP2, CP1	L'utente digita una password troppo corta	La password deve essere tra 8 e 20 caratteri.
TC_CPW_3	VP1, NP1, CP2	L'utente digita correttamente la nuova password ma sbaglia a ripeterla nel campo conferma	Le due password non coincidono.
TC_CPW_4	VP1, NP3, CP1	L'utente prova a rimettere la stessa password provvisoria come "Nuova".	La nuova password deve essere diversa dalla provvisoria.

9.7 Funzionalità: Compilazione e Salvataggio Referto (UC9)

Si verifica che il medico possa salvare un referto solo se è presente del contenuto testuale e se la visita si trova nello stato corretto ("Effettuata/In Attesa di Referto").

Parametro: Contenuto Referto [C]

- Testo valido (Non vuoto) [propertyContentOk]
- Testo vuoto o nullo [error]

Parametro: Stato Visita (Pre-condizione) [S]

- Stato "EFFETTUATA" (Pronta per essere refertata) [propertyStateOk]
- Stato "PRENOTATA", "CONCLUSA" o "ANNULLATA" (Non refertabile) [error]

Codice Test	Combinazione	Descrizione Scenario	Esito Atteso
TC_REF_1	C1, S2	Tentativo di refertare una visita futura ("PRENOTATA") o già chiusa ("CONCLUSA")	Impossibile refertare una visita in questo stato
TC_REF_2	C:2, S:1	Tentativo salvataggio referto vuoto	Il contenuto del referto è obbligatorio
TC_REF_3	C:1, S1	Il medico scrive l'esito clinico. La visita è stata appena eseguita (stato "EFFETTUATA").	Lo stato della visita diventa "CONCLUSA".